Phase 2 Preparation Document

Project: Virtualfactor IT CMDB
Repository: https://github.com/jallamasc/vf-cmdb
Date: 2026-07-23
User: Alejandro (jallamasc), Virtualfactor, Bogotá, Colombia
Predecessor: Phase 1 complete — see `docs/PHASE_1_COMPLETION.md`

---

0. Starting State (post Phase 1)

- **Branch to base Phase 2 on:** merge PR #3 (`feature/phase-1-quick-add-rack-hierarchy`) into `master` first, then branch `feature/phase-2-ipam-by-site` .
- **Schema now includes:** physical hierarchy (`datacenters` , `datacenter_floors` , `rooms` , `rack_types`), the naming engine (`abbreviation_registry` , `case_enforcement` , `trim_mode`), and device `name_prefix` / `sequence_number` .
- **Existing IPAM tables (pre-Phase 2):**
- `vlans` — `id` , `vlan_id` (**globally** `unique`), `name` , `description` , `zone` , `site_id` (nullable FK).
- `subnets_ipv4` — `id` , `vlan_id` FK, `network_cidr` (CIDR), `gateway` , `range_from` , `range_to` , `expansion_ceiling` , `description` .
- `subnets_ipv6` — analogous.
- `subnet_role_assignments` — `id` , `subnet_ipv4_id` / `subnet_ipv6_id` , `role` , `slot_number` , `ipv4_address` , `ipv6_address` , `assigned_device_id` , `assigned_device_table` , `notes` . **← natural home for reserved IPs.**
- `ip_assignments` — active/reserved/deprecated host assignments.
- **Existing IPAM endpoints:** `GET /ipam/subnets/{id}/next-ip` , `GET /ipam/subnets/{id}/utilization` (both in `routers/special.py`).

---

1. Phase 2 Objective

Two parallel work streams:

1.1 Complete the Rack View Upgrade (deferred from Phase 1 original scope)

Hierarchy tables exist, but the **RackView page** still has no hierarchy filters,
no multi-rack layout, and no realistic SVG diagrams. Phase 2 delivers:
- Datacenter/Floor/Rack cascading filter dropdowns.
- Multi-rack side-by-side layout.
- Professional **SVG-based rack elevation diagrams** (Visio-like, numbered U positions,
device rectangles with colors/labels).

1.2 IPAM by Site (new requirements from user)

Turn IPAM into a **site-scoped** system with **manageable reserved-IP pools** per network segment, and make reservation placement (from the last usable IP downward, gap-aware) fully configurable from the UI.

---

2. Work Stream A — Rack View Upgrade (deferred from Phase 1)

2.1 Current State

- `RackView.tsx` exists but shows only basic front-elevation diagrams (colored boxes for device types).
- Can filter “All racks” or a specific rack.
- **No datacenter/floor hierarchy** in the UI (the tables exist in the DB but aren’t used by RackView).
- **No realistic rack visualization** (no U numbering, no proper scaling, no professional appearance).

2.2 Requirements (from original Phase 1 scope)

A. Hierarchy Filter UI

Three cascading dropdowns at the top of the Rack View page:
1. **Datacenter** (all datacenters for the selected site + “All Datacenters”)
2. **Floor** (floors for the selected datacenter + “All Floors”)
3. **Rack** (racks for the selected floor + “All Racks”)

Selection updates the rack display below. When “All Racks” or multiple racks are in scope, display them side-by-side (responsive grid, 2-3 columns on desktop).

B. Multi-Rack Layout

- When viewing multiple racks, display them in a **responsive grid** (flexbox or CSS grid).
- Each rack labeled with **Datacenter / Floor / Rack name** above it.
- Scroll horizontally/vertically as needed.

C. Realistic SVG Rack Diagrams

Replace the current colored-box diagrams with professional **SVG-based rack elevation diagrams** similar to Microsoft Visio rack stencils:

Visual Requirements:
- Vertical rectangle representing the rack.
- **U positions numbered** (1U at bottom, ascending upward to 16U, 32U, 42U, or 48U depending on `rack.total_units`).
- Rails on the sides.
- **Devices as colored rectangles** spanning their U height (`device.units`).
- Device labels inside rectangles (truncate if needed).
- Empty U slots shown as light gray background.
- Color-code by device type (reuse existing `TYPE_COLORS` mapping).

SVG Specifications:
- Width: ~300px (rack front width).

- Height: dynamic based on U count (1U ≈ 19px; 42U ≈ 800px).
- ViewBox: `0 0 300 {height}`.
- Elements:
- Outer frame: rectangle stroke.
- Rails: two vertical lines on sides.
- U position labels: text on left side, right-aligned, every 1U.
- Device rectangles:
- X: 50px (after rail), Width: 200px.
- Y: calculated from bottom up based on U position.
- Height: `device.units * 19px`.
- Fill: color from TYPE_COLORS (convert Tailwind classes to hex).
- Stroke: darker shade of same color.
- Device labels: text centered in rectangle.

2.3 Backend Work (Rack View)

None required — hierarchy tables and device tables already exist. The existing `GET /racks`, `GET /datacenters`, `GET /datacenter-floors`, `GET /rooms`, `GET /rack-units` endpoints provide all needed data.

2.4 Frontend Work (Rack View)

Files to Create:
- `frontend/src/components/RackDiagramSVG.tsx` — the SVG rack visualization component.

Files to Modify:
- `frontend/src/pages/RackView.tsx` — complete rewrite:
- Add hierarchy filter dropdowns (site → datacenter → floor → rack).
- Multi-rack grid layout.
- Replace current diagram with `<RackDiagramSVG />`.

Component Structure:

```
RackView.tsx
├── HierarchyFilters (dropdowns for Site/DC/Floor/Rack)
├── RackGrid (responsive grid container)
│   └── RackCard (one per rack, includes DC/Floor/Rack label)
│       └── RackDiagramSVG (the actual SVG visualization)
└── Legend (device type colors)
```

---

3. Work Stream B — IPAM by Site (new requirements)

3.1 R1 — IPAM is scoped by Site

> "IPAM should be by Site. E.g. Site Korriban has VLANs 100, 101, 66, 67, …;
> those VLANs and network segments must not be used on other sites."

Interpretation / rules:
- Every VLAN and every subnet/segment **belongs to exactly one site**.
- A given VLAN number / network segment **cannot be reused on another site** (no cross-site duplication).
- All IPAM views, pickers, and allocation operations are filtered by the selected site.

Schema impact:
- `vlans.site_id` → make **NOT NULL** (every VLAN must belong to a site).
- Replace the **global** `unique` on `vlans.vlan_id` with a composite `UNIQUE (site_id, vlan_id)` **plus** a guard that the same `vlan_id` is not present under a different site (enforces "not used on other sites"). Two options, to decide in implementation:
- (a) Keep a global uniqueness guard on `vlan_id` (simplest; literally prevents reuse anywhere) — matches the user's words most directly.
- (b) `UNIQUE (site_id, vlan_id)` + application check rejecting a `vlan_id` that already exists in any other site.
→ **Leaning (a)** because the requirement explicitly says a VLAN must not be used on other sites at all. Confirm during kickoff.
- `subnets_ipv4` / `subnets_ipv6`: add `site_id` (derive/validate from the parent VLAN's site; deny cross-site overlap). Add an **overlap check**: a CIDR may not overlap another segment on the same site, and (per R1) may not be reused on another site.

3.2 R2 — Configurable reserved IPs per segment

> "Each segment normally reserves IPs for firewalls, switches, etc. This must be configurable/manageable in the app. E.g. `192.168.0.0/24` but only 7 IPs reserved for special devices/services."

Design:
- A segment gets a configurable **reserved pool**: a count (e.g. 7) and a set of **reservation entries**, each with a role/label (firewall, core switch, gateway, VIP, …) and the resolved IP.
- Reuse **`subnet_role_assignments`** as the reservation store (it already has `role`, `slot_number`, `ipv4_address`). Add if needed:
- `label` (free text distinct from `role`), `is_locked` (protect from auto-realloc),
- and on `subnets_ipv4/6`: `reserved_count` (int) + `reservation_anchor` (`from_end` default | `from_start`) + `reservation_direction`.
- Reserved IPs are **excluded from `next-ip`** allocation (currently `next-ip` only skips `ip_assignments`; it must also skip `subnet_role_assignments`).

3.3 R3 — Reservation placement: from the last IP, gap-aware & configurable

> "Reserved IPs are taken from the last IP and so on; that should be configurable and gaps should be taken into account."

Design:
- Default anchor = **`from_end`**: assign reservations starting at the last usable host address and walk downward (…254, …253, …252 for a /24, skipping network & broadcast). Configurable to `from_start` per segment.
- **Gap-aware:** when computing the next reservation slot, skip IPs already taken by either a reservation or an `ip_assignment`; reuse freed gaps (same UX pattern as the Phase 1 naming gap helper — prompt: "gap at .251 available — reuse or take next .249?").
- New endpoints:
- `GET /ipam/subnets/{id}/reservations` — list current reservations + computed free slots.
- `POST /ipam/subnets/{id}/reservations` — add a reservation (role/label; optional explicit IP or auto

from anchor).
- `DELETE /ipam/subnets/{id}/reservations/{res_id}` — free a reservation (creates a gap).
- `GET /ipam/subnets/{id}/next-reserved` — compute the next reservation IP per anchor+gaps.
- Extend `GET /ipam/subnets/{id}/next-ip` to **exclude reserved** addresses.

---

3. Proposed Schema Changes (draft — finalize in implementation)

```
-- VLANs: enforce site scoping
ALTER TABLE vlans ALTER COLUMN site_id SET NOT NULL;
-- Option (a): keep vlan_id globally unique (prevents reuse on any other site)
--   (already unique today) — add composite for query ergonomics:
ALTER TABLE vlans ADD CONSTRAINT uq_vlan_site_vlanid UNIQUE (site_id, vlan_id);

-- Subnets: attach to a site, support reserved pools
ALTER TABLE subnets_ipv4 ADD COLUMN site_id INTEGER REFERENCES sites(id);
ALTER TABLE subnets_ipv4 ADD COLUMN reserved_count INTEGER DEFAULT 0;
ALTER TABLE subnets_ipv4 ADD COLUMN reservation_anchor VARCHAR(10) DEFAULT 'from_end'
    CHECK (reservation_anchor IN ('from_end','from_start'));
-- (mirror onto subnets_ipv6)

-- Reservations reuse subnet_role_assignments; add:
ALTER TABLE subnet_role_assignments ADD COLUMN label VARCHAR(80);
ALTER TABLE subnet_role_assignments ADD COLUMN is_locked BOOLEAN DEFAULT FALSE;
```

Migration: new **`0004_ipam_by_site.py`**, guarded/idempotent (same helper pattern as `0003`). Backfill `subnets_ipv4.site_id` from the parent VLAN's site.

3.4 Backend Work (IPAM)

- **Models** (`models.py`): site scoping + new columns above; relationships `Site → Vlan → Subnet → Reservation`.
- **Registry** (`registry.py`): ensure `vlans` , `subnets-ipv4` , `subnets-ipv6` , `subnet-role-assignments` are addressable slugs for CRUD if not already.
- **Validation** (`crud.py`): cross-site VLAN/segment reuse guard; CIDR overlap guard per site; keep clean 409/422 responses.
- **IPAM service** (`routers/special.py` or new `routers/ipam.py`):
- reservation CRUD + `next-reserved` (anchor + gap-aware),
- update `next-ip` / `utilization` to account for reservations,
- all IPAM list endpoints accept a `site_id` filter.
- **Seed** (`seed.py` / `seed_subnets.json`): attach existing sample subnets to a site; demonstrate a reserved pool (e.g. 7 reserved from the top of a /24).

3.5 Frontend Work (IPAM)

- **New page** **`IPAM.tsx`** (site-scoped):
- Site selector at top → cascades to that site's VLANs and segments only.
- VLAN list/quick-add (inline collapsible form, Phase 1 pattern).
- Segment (subnet) list/quick-add with `reserved_count` + anchor selector.

- **Reservation manager per segment:** table of reserved IPs (role/label/IP/locked), add/remove, and a **gap-aware "next reserved IP" helper** mirroring `SequenceGapHelper` UX.
- Utilization bar (reused/extended from existing `utilization` endpoint).
- **`api.ts`:** add `reservations`, `addReservation`, `removeReservation`, `nextReserved`, and `site_id`-filtered IPAM list calls.
- **Nav** (`Layout.tsx`): add "IPAM" under a Networking section.

---

4. Open Questions for Kickoff

1. **VLAN reuse rule:** strict global (option a) or per-site unique with cross-site reject (option b)? (Doc leans **a** per your wording.)
2. **Segment overlap:** forbid overlapping CIDRs within a site outright, or warn only?
3. **Reserved default count:** global default (e.g. 0) or a per-segment prompt on creation? Any standard roles to pre-populate (gateway, firewall, core-switch, VIP)?
4. **Anchor default:** confirm `from_end` (last usable IP downward) as the default.
5. **IPv6:** apply the same reservation model to `subnets_ipv6` now, or IPv4-first?
6. **Gateway convention:** is the gateway always the first or last usable IP, and should it be auto-created as a locked reservation?

---

5. Success Criteria

Rack View (Stream A)

✅ Cascading hierarchy filters (Site → Datacenter → Floor → Rack) working.
✅ Multi-rack side-by-side responsive grid layout.
✅ Professional SVG rack diagrams with numbered U positions, device rectangles, colors, labels.
✅ Visual appearance comparable to Visio rack stencils.

IPAM (Stream B)

✅ VLANs and segments are owned by a site and cannot be duplicated across sites.
✅ Each segment exposes a configurable reserved-IP pool, managed from the UI.
✅ Reservations default to allocation from the last usable IP downward, are gap-aware, and the anchor/direction is configurable per segment.
✅ `next-ip` never returns a reserved address; utilization reflects reservations.

General

✅ No regressions; `alembic upgrade head`, `python -m app.seed`, and `npm run build` all succeed.

---

6. Git Workflow

1. Merge PR #3 to `master`.
2. `git checkout -b feature/phase-2-rack-ipam`.

3. Commits (suggested order):
   - (1) Frontend: RackDiagramSVG component
   - (2) Frontend: RackView rewrite with hierarchy filters + multi-rack layout
   - (3) Backend: IPAM schema+migration (0004_ipam_by_site.py)
   - (4) Backend: IPAM service/endpoints (reservation CRUD, site scoping)
   - (5) Frontend: IPAM page (VLANs, segments, reservations)
   - (6) Seed + docs update
4. Open PR against `master`; update `docs/` on completion (`PHASE_2_COMPLETION.md`).

7. Deployment Reminder (carried from Phase 1 incident)

Use **one** lifecycle manager per host (compose **or** Quadlet, never both). If a volume-store error recurs ("more than one result for volume name"), run `./deploy-podman.sh recover` (backs up on-disk data first) **before** any `podman system reset`.